

Page 1 — Introduction to AI and Language

Artificial intelligence allows computers to perform tasks that normally require human intelligence. Language models process and generate human language. Before a model can work with text, the text must be represented in a form that a computer can process. Natural language contains words, punctuation, numbers, symbols, abbreviations, and many writing styles. Tokenization is one important step in creating this representation.

Page 2 — What Is Tokenization?

Tokenization is the process of breaking text into smaller units called tokens. A token may be a complete word, part of a word, punctuation, or a special symbol, depending on the tokenizer. Consider: Rachit is learning RAG. A simple tokenizer might produce tokens corresponding to the words and punctuation. Modern language-model tokenizers often use subword techniques, so uncommon or long words can be divided into smaller pieces.

Page 3 — Words Are Not Always Tokens

One important idea is that one word does not necessarily equal one token. Common words may frequently correspond to one token, while uncommon words can be split into multiple pieces. A word containing a prefix, suffix, unusual spelling, or punctuation may also be divided. Therefore, token counts cannot always be predicted by simply counting words. The exact result depends on the tokenizer and vocabulary used by the model.

Page 4 — Punctuation, Numbers, and Symbols

Tokenizers process punctuation, numbers, and symbols too. Try experimenting with: Hello! How are you? 123 4567 3.14159 2026 email@example.com #learning AI/RAG Python_101. A decimal number may be divided into several pieces. An email address can contain several segments. Symbols such as #, /, _, and @ can affect tokenization. Technical text and ordinary sentences can therefore have very different token patterns.

Page 5 — Tokenization and PDFs

A PDF can contain text, images, fonts, and layout information. In a RAG project, a PDF loader can extract textual content so it can be processed further. A typical workflow is PDF, then PDF Loader, then Documents, then Text Splitter, then Chunks. The extracted text may contain line breaks or spacing that differ from the visual PDF. After loading, a token-based splitter can divide the document according to token limits.

Page 6 — Character Splitting vs Token Splitting

There are several ways to divide a document. Character-based splitting considers characters and separators, while token-based splitting considers tokens generated by a tokenizer. A character splitter might use 500 characters, while a token splitter might use 500 tokens. These measurements are not equivalent. The relationship between characters, words, and tokens changes depending on the text.

Page 7 — Chunk Size and Overlap

Chunk size controls approximately how much content is placed into each chunk. Chunk overlap determines how much content from the previous chunk is repeated in the next chunk. For example, with chunks of 100 tokens and an overlap of 10 tokens, the next chunk reuses some information from the previous chunk. Overlap can preserve context near chunk boundaries. Very small chunks may lose context, while very large chunks may contain unnecessary information.

Page 8 — Tokenization in RAG

Retrieval-Augmented Generation, commonly called RAG, combines document retrieval with language generation. A RAG system may load documents, split them into chunks, create embeddings, store them in a vector database, and retrieve relevant chunks when a user asks a question. Tokenization matters because language models have token-based context limits. Understanding tokens helps developers reason about context size, cost, latency, and document processing.

Page 9 — Practical Experiments

Try this text with your tokenizer: Learning AI is an exciting journey. RAG can retrieve useful information from documents. Don't stop learning! Machine-learning systems process data differently from traditional software. Try numbers: 10, 20, 30, 1000. Try symbols: @ # \$ % & * / _ + =. Try repeated words: token token token token. Change the text and observe how the token count changes. Add long words, punctuation, numbers, emojis, and code-like expressions.

Page 10 — Final Practice and RAG Pipeline

A useful mental model is: raw document → extracted text → tokens → chunks → embeddings → vector store → retrieval → prompt → language model → answer. For practice, load this PDF using PyPDFLoader, inspect the Documents, use TokenTextSplitter with different chunk sizes, print the number of chunks, and display every chunk. Experiment with overlap values such as 0, 5, 10, and 20. The goal is to understand what happens to the text at every stage.